Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance. In this experiment, all five architectures were implemented and evaluated on the CIFAR-10 dataset: LeNet-5 and AlexNet were trained from scratch (no ImageNet-pretrained weights exist for these historic architectures), while VGG16, GoogleNet (approximated using InceptionV3, the closest Inception-family model available in the Keras applications library, since the original GoogleNet/Inception-v1 checkpoint is not distributed with Keras) and ResNet50 were fine-tuned using transfer learning with ImageNet-pretrained weights.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1, 3×3, 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

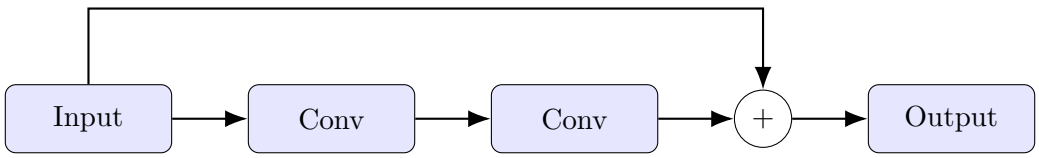


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

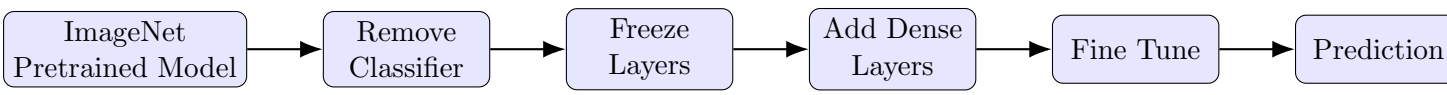


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range [0,1].
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Mandatory Plots

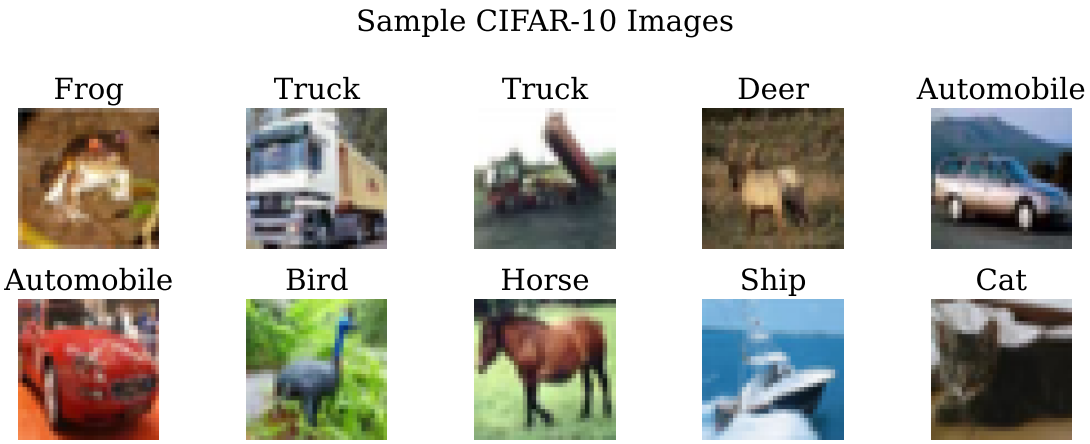


Figure 3: Sample CIFAR-10 Images

Inference: Dataset preview indicates proper loading and normalization of the data. The low resolution (32×32) is the reason behind the difficulty in differentiating fine classes like cat, dog, and deer visually.

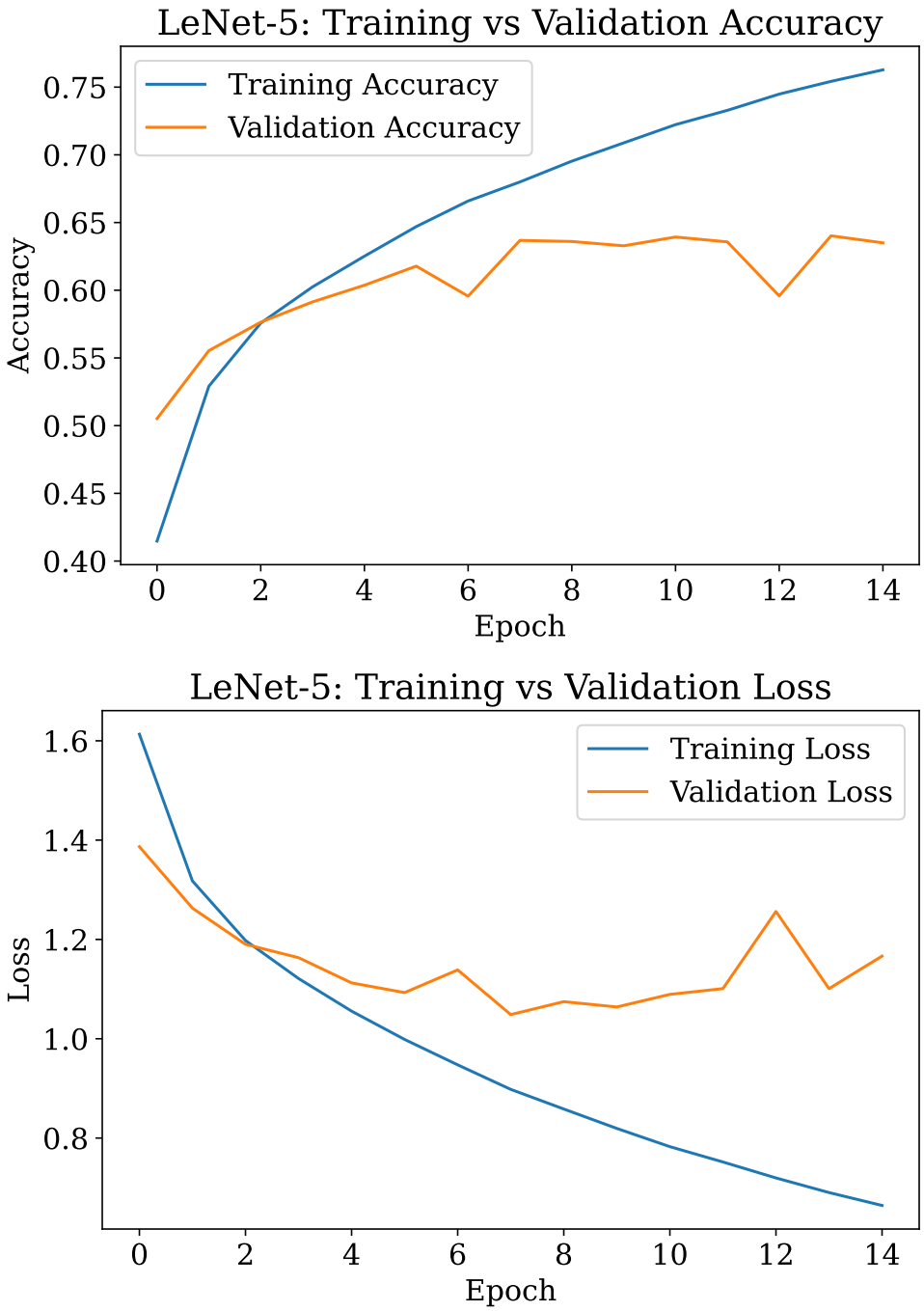


Figure 4: LeNet-5: Training/Validation Accuracy and Loss

Inference: Accuracy during training increases to ∼76%, but the validation accuracy remains constant between 63-64%, and validation loss starts increasing after epoch 8 – the classic sign that the model is overfitting on a more complicated dataset than it was initially trained for (handwritten digits).

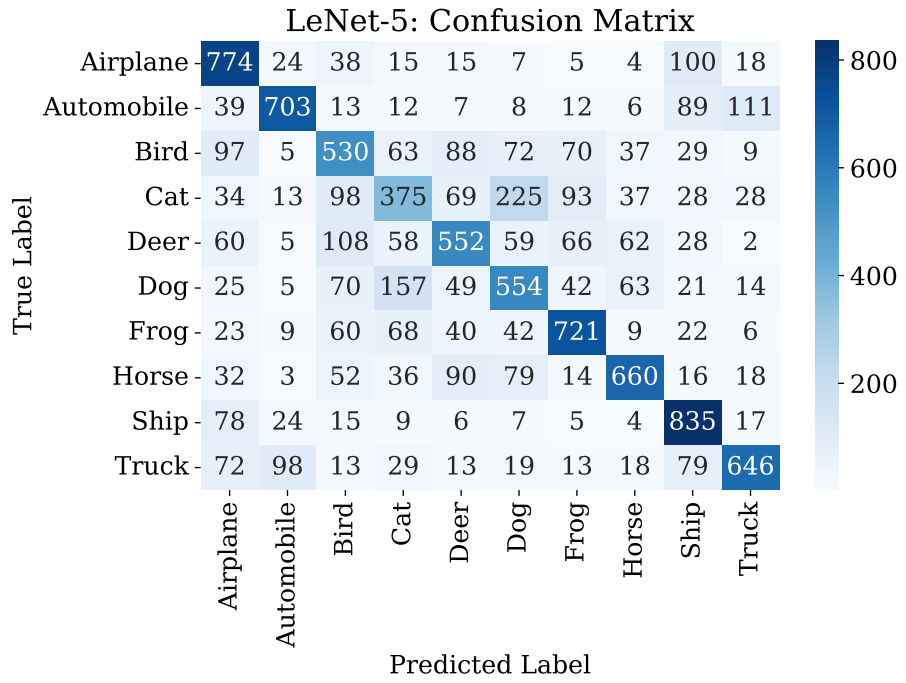


Figure 5: LeNet-5: Confusion Matrix

Inference: Airplane, automobile, ship, and frog are strongly classified, whereas the classification of the cat and dog categories is highly confused (225 and 157 misclassifications respectively).

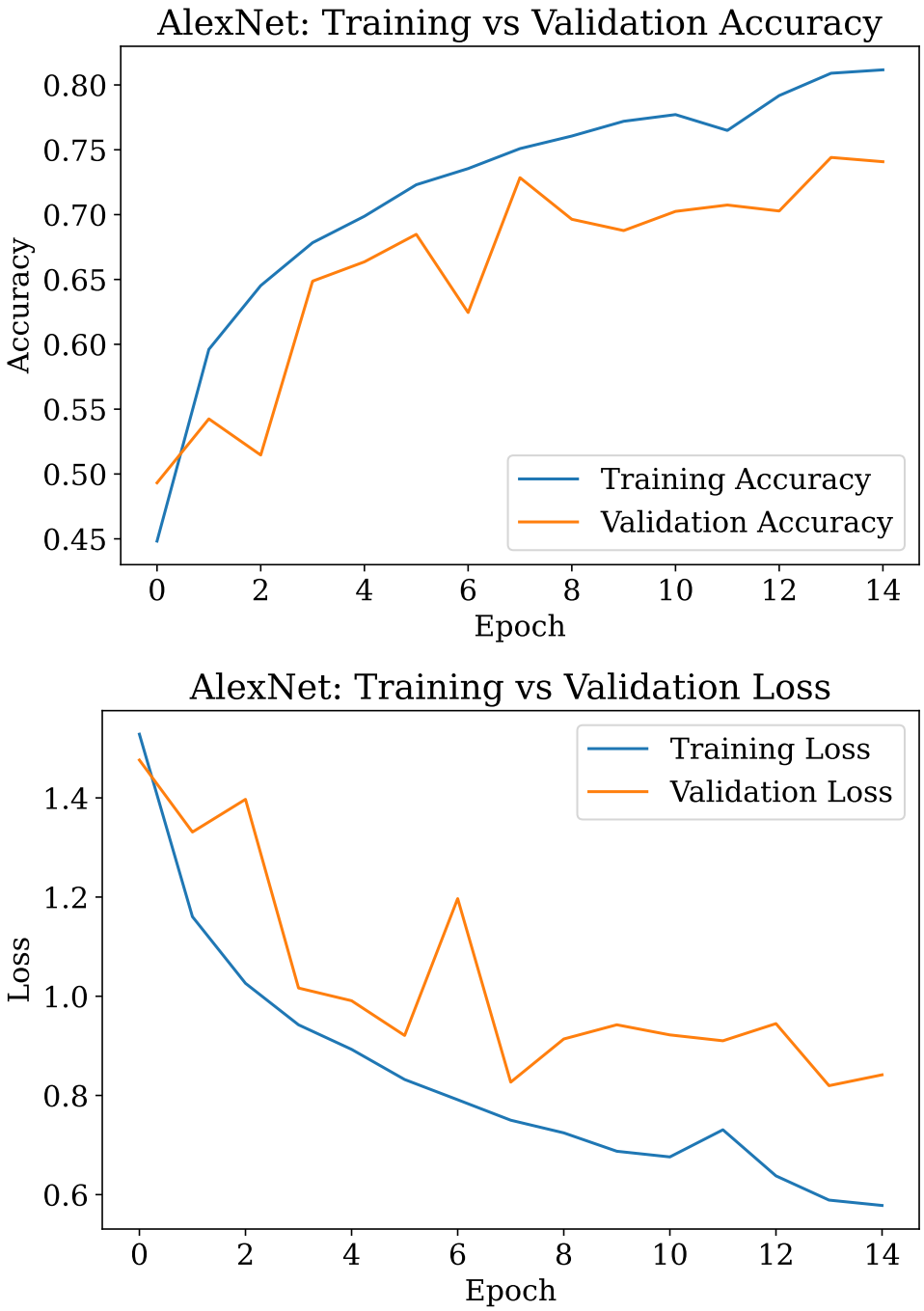


Figure 6: AlexNet: Training/Validation Accuracy and Loss

Inference: Training accuracy increases above 80%, whereas validation accuracy lows and plateaus around 74%, and validation loss drops and goes back up above epoch 8 due to the model learning too well on the training set, once the model is deep enough to do so.

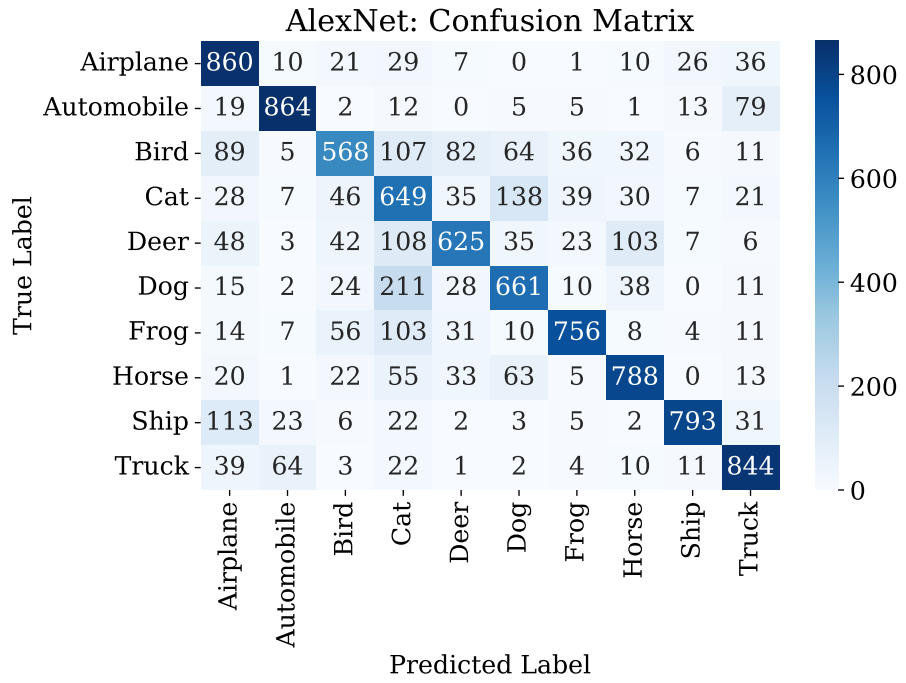


Figure 7: AlexNet: Confusion Matrix

Inference: AlexNet model exhibits the most prominent diagonal from the two from-scratch models (860 airplane, 864 automobile, 844 truck correct), proving that the deep customized architecture can provide more discriminative features than LeNet-5, while the cat/dog pair is still the worst (138/211 confused).

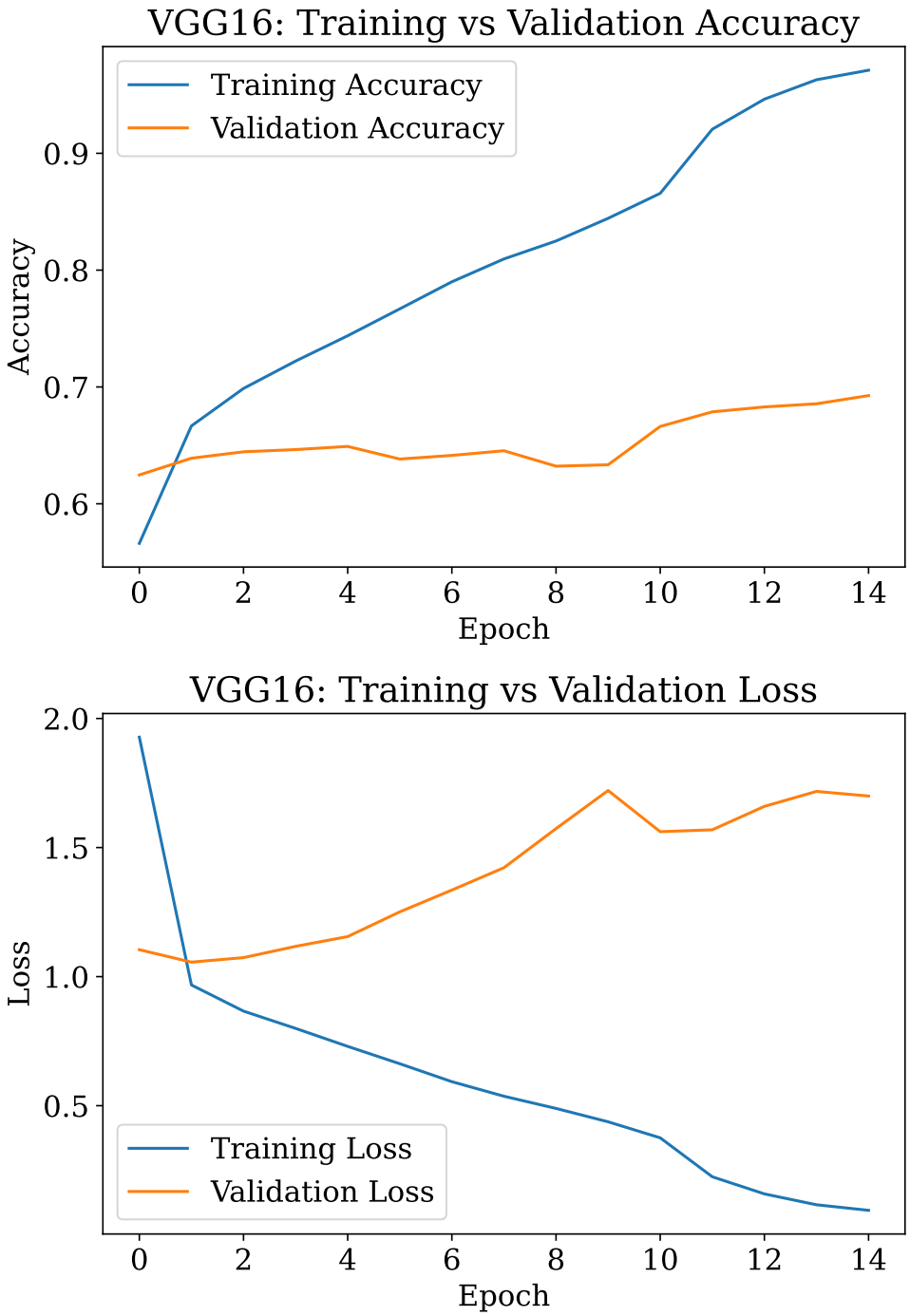


Figure 8: VGG16: Training/Validation Accuracy and Loss

Inference: With transfer learning, VGG16 is able to have a much faster increase in training accuracy (>97%), but validation accuracy is increased slightly to ∼69%; the increasing difference between training and validation loss after fine-tuning shows that overfitting occurs because the unfrozen block becomes trained to the small CIFAR-10 dataset.

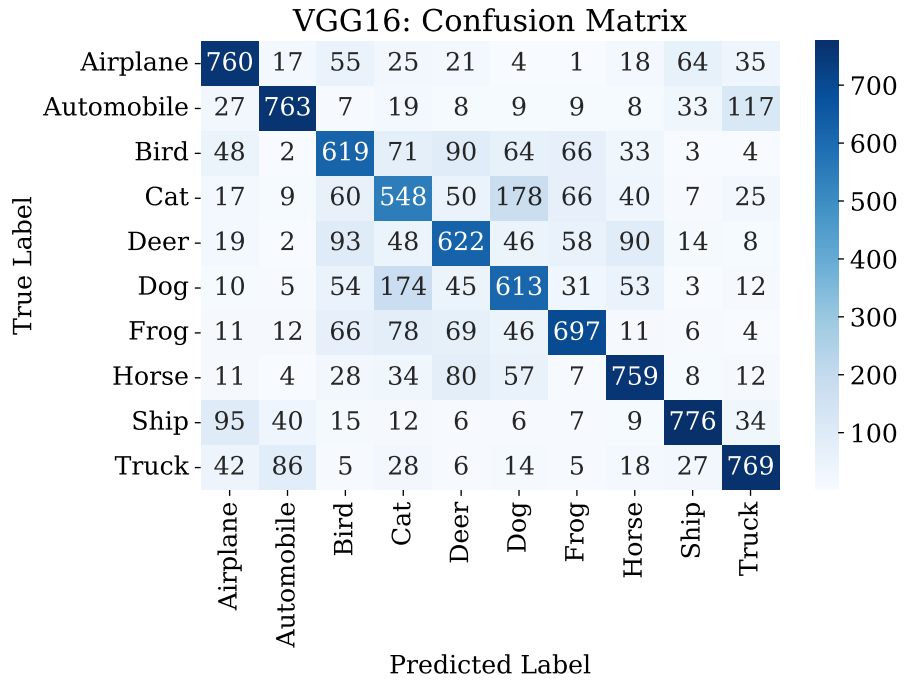


Figure 9: VGG16: Confusion Matrix

Inference: VGG16 demonstrates the most prominent diagonal of all three transfer learning models (776 ship, 769 truck, 763 airplane correct) since it has a very deep convolutional architecture using 3×3 filters to capture texture details even from tiny input sizes.

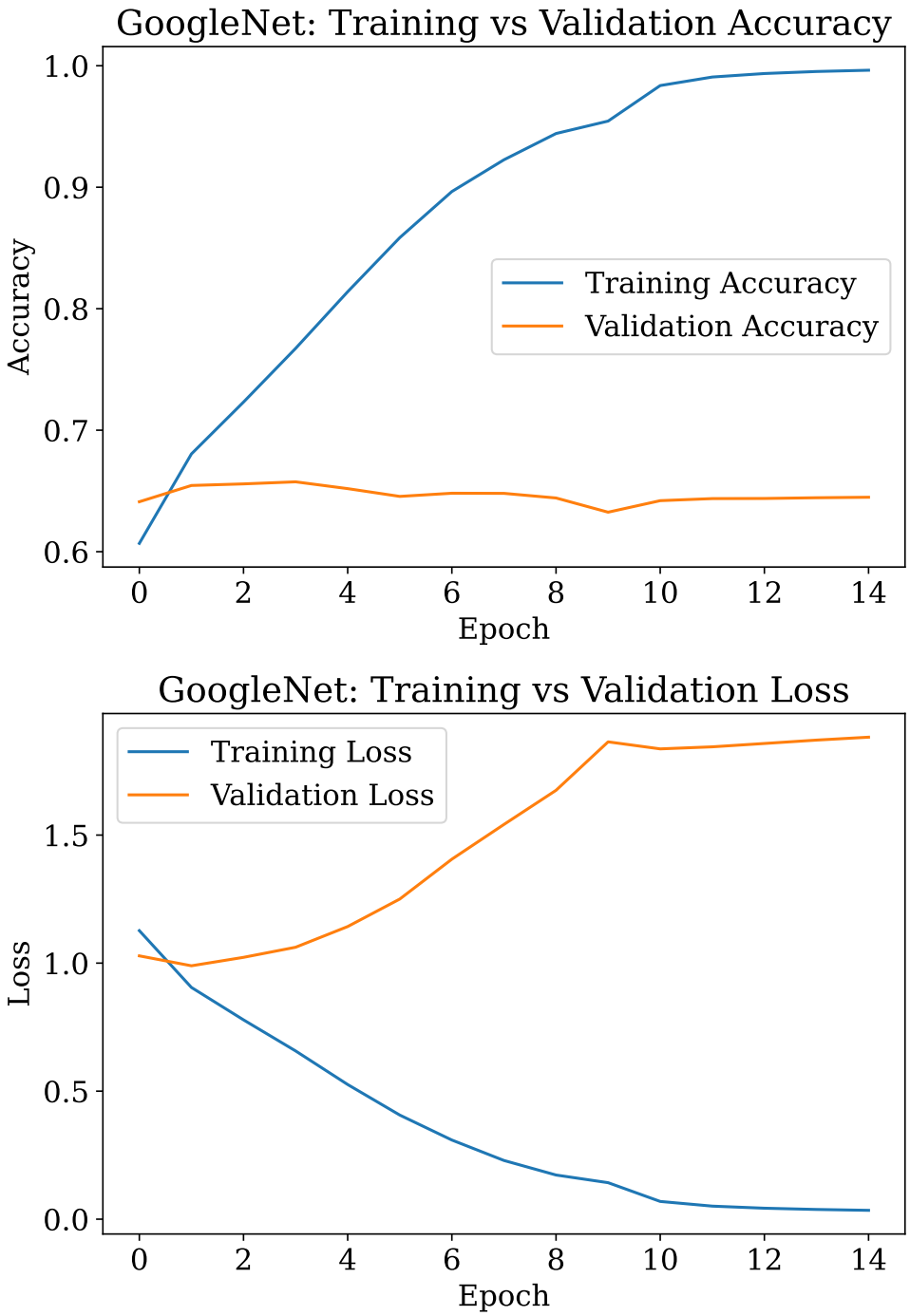


Figure 10: GoogleNet (InceptionV3): Training/Validation Accuracy and Loss

Inference: Training accuracy reaches 99% whereas validation accuracy remains close to 64–65%, while validation loss increases dramatically – the highest train/validation difference between all five models, signifying the tendency of InceptionV3's deep backbone to overfit fastest among all models.

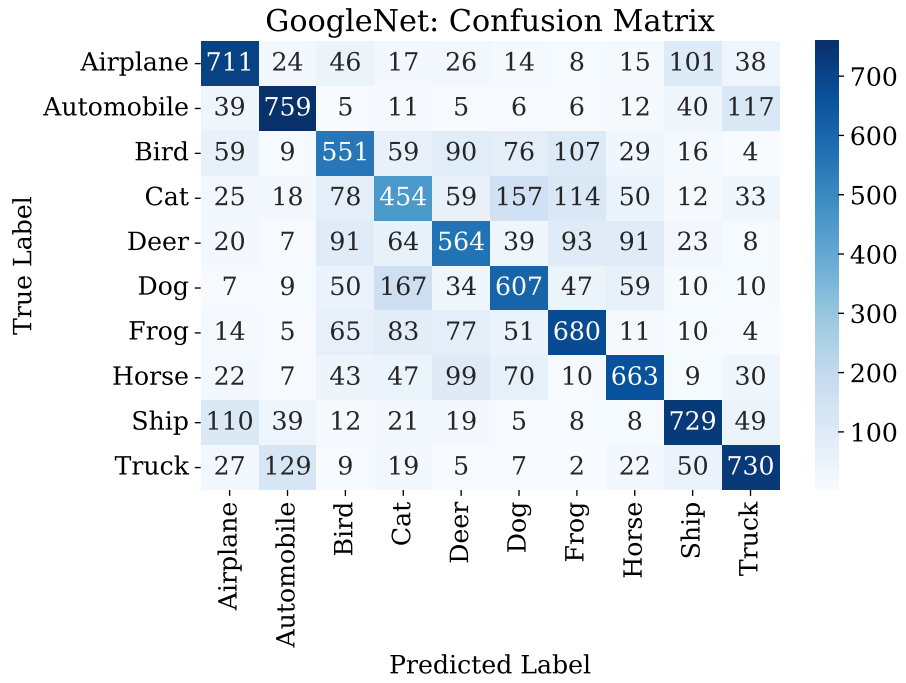


Figure 11: GoogleNet (InceptionV3): Confusion Matrix

Inference: Dog/cat confusion exists (157/167), even though there is a pretrained backbone, and the level of diagonal dominance is less than both VGG16 and ResNet50, indicating that the 75×75 upsampling necessary for InceptionV3 can create some artifacts.

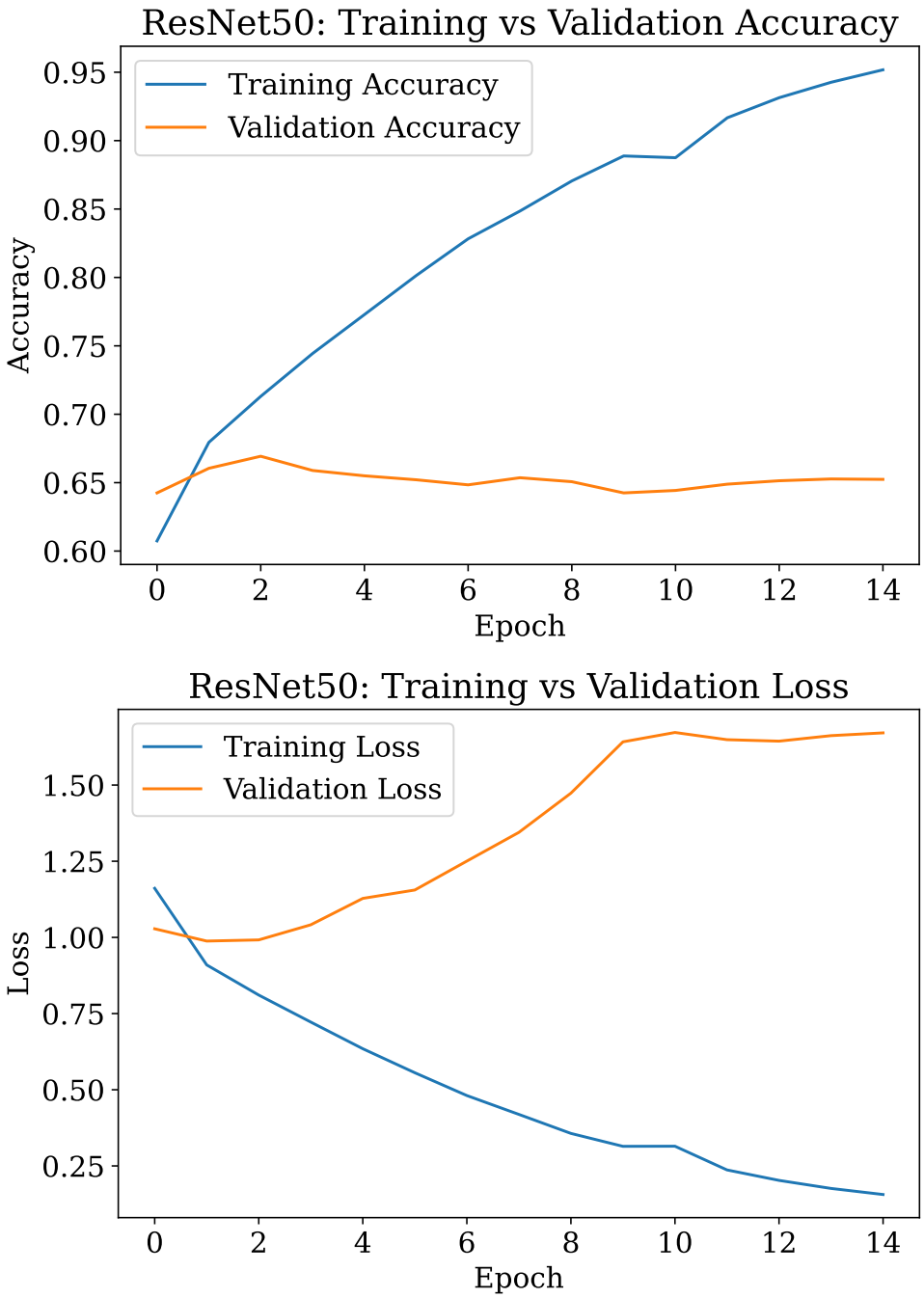


Figure 12: ResNet50: Training/Validation Accuracy and Loss

Inference: However, the ResNet50 architecture follows a trend similar to that of GoogleNet – fast convergence in training accuracy (around $\sim$ 95%) together with relatively low validation accuracy (around $\sim$ 65%), but with much smoother curves in terms of loss function, indicating that the residual connections help optimize the problem.

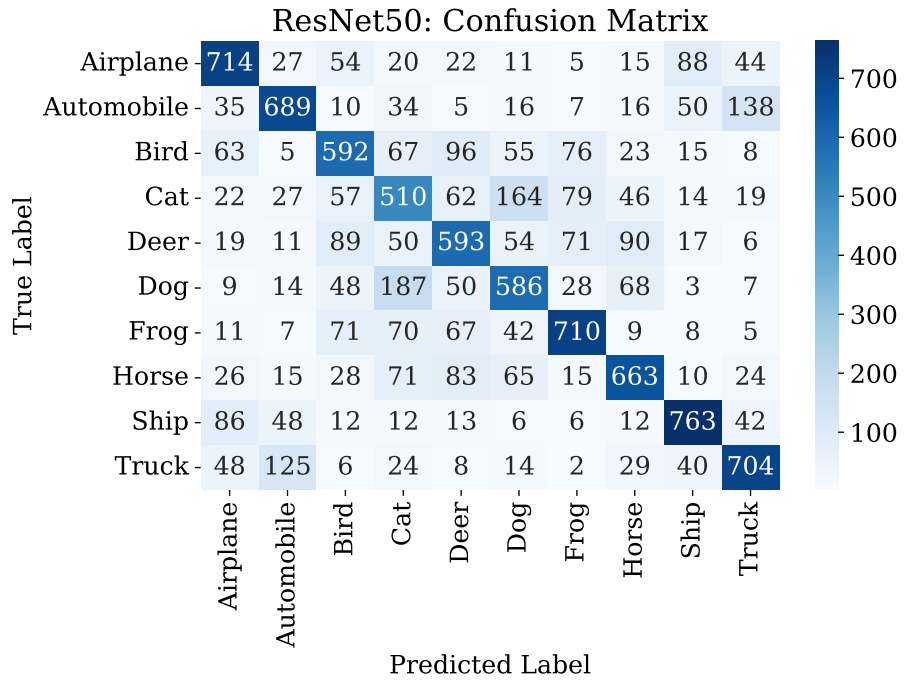


Figure 13: ResNet50: Confusion Matrix

Inference: Cat and dog classification difficulty similar to other approaches (164/187 wrong), but ships and airplanes are very well classified (763 and 714 correct). This demonstrates that the approach is good at separating features even when early layers are not fine-tuned.



Figure 14: ResNet50: Sample Misclassified Images (Optional)

Inference: These misclassifications are primarily between plausible pairings (dog→frog, deer→horse, airplane→bird), indicating that the mistakes occur due to confusion from the pose and background in low-resolution images, rather than a model flaw

18. Results

18.1 Performance Metrics (Primary Model: VGG16)

Metric	Value
Training Accuracy	97.11%
Testing Accuracy	69.26%
Precision (macro avg)	69.49%
Recall (macro avg)	69.26%
F1-score (macro avg)	69.34%
Training Time	312.3 sec
Total Parameters	14,848,586

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time (s)
LeNet-5	83,126	63.50	118.7
AlexNet	6,977,866	74.08	397.8
VGG16	14,848,586	69.26	312.3
GoogleNet	22,329,898	64.48	677.6
ResNet50	24,114,826	65.24	640.1

18.3 Per-Class Classification Reports

LeNet-5 (Accuracy = 63.50%)

Class	Precision	Recall	F1-score
Airplane	0.63	0.77	0.69
Automobile	0.79	0.70	0.74
Bird	0.53	0.53	0.53
Cat	0.46	0.38	0.41
Deer	0.59	0.55	0.57
Dog	0.52	0.55	0.53
Frog	0.69	0.72	0.71
Horse	0.73	0.66	0.69
Ship	0.67	0.83	0.74
Truck	0.74	0.65	0.69

AlexNet (Accuracy = 74.08%)

Class	Precision	Recall	F1-score
Airplane	0.69	0.86	0.77
Automobile	0.88	0.86	0.87
Bird	0.72	0.57	0.63
Cat	0.49	0.65	0.56
Deer	0.74	0.62	0.68
Dog	0.67	0.66	0.67
Frog	0.86	0.76	0.80
Horse	0.77	0.79	0.78
Ship	0.91	0.79	0.85
Truck	0.79	0.84	0.82

VGG16 (Accuracy = 69.26%)

Class	Precision	Recall	F1-score
Airplane	0.73	0.76	0.75
Automobile	0.81	0.76	0.79
Bird	0.62	0.62	0.62
Cat	0.53	0.55	0.54
Deer	0.62	0.62	0.62
Dog	0.59	0.61	0.60
Frog	0.74	0.70	0.72
Horse	0.73	0.76	0.74
Ship	0.82	0.78	0.80
Truck	0.75	0.77	0.76

GoogleNet (InceptionV3) (Accuracy = 64.48%)

Class	Precision	Recall	F1-score
Airplane	0.69	0.71	0.70
Automobile	0.75	0.76	0.76
Bird	0.58	0.55	0.57
Cat	0.48	0.45	0.47
Deer	0.58	0.56	0.57
Dog	0.59	0.61	0.60
Frog	0.63	0.68	0.66
Horse	0.69	0.66	0.68
Ship	0.73	0.73	0.73
Truck	0.71	0.73	0.72

ResNet50 (Accuracy = 65.24%)

Class	Precision	Recall	F1-score
Airplane	0.69	0.71	0.70
Automobile	0.71	0.69	0.70
Bird	0.61	0.59	0.60
Cat	0.49	0.51	0.50
Deer	0.59	0.59	0.59
Dog	0.58	0.59	0.58
Frog	0.71	0.71	0.71
Horse	0.68	0.66	0.67
Ship	0.76	0.76	0.76
Truck	0.71	0.70	0.71

18.4 Cross-Model Comparison Plots

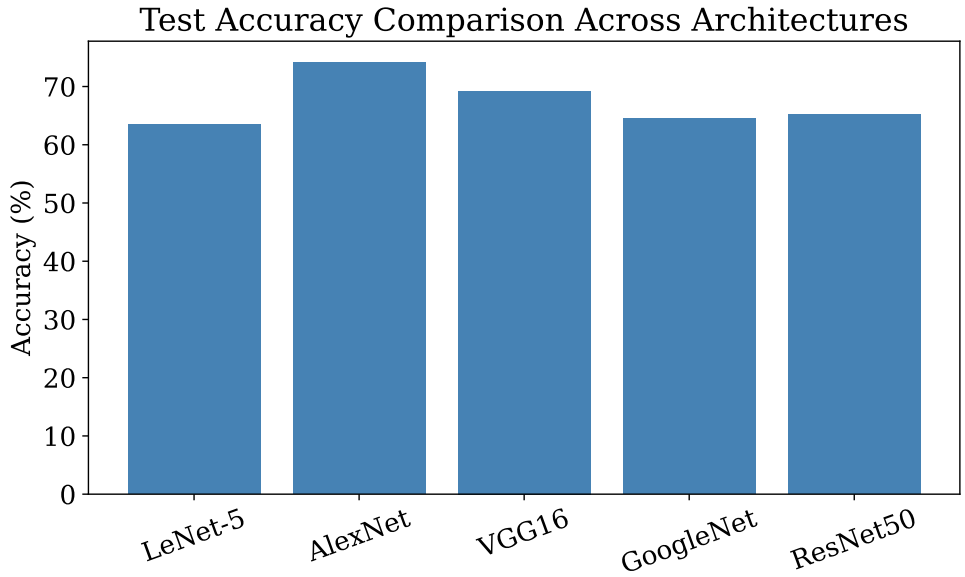


Figure 15: Test Accuracy Comparison Across Architectures

Inference: All of the five models have similar results of 63 to 74% on CIFAR-10, but surprisingly, the from scratch AlexNet is the best out of the three transfer learning models whereas transfer learning outperforms LeNet-5. It shows that depth along with the pretrained features work well when fine-tuned.

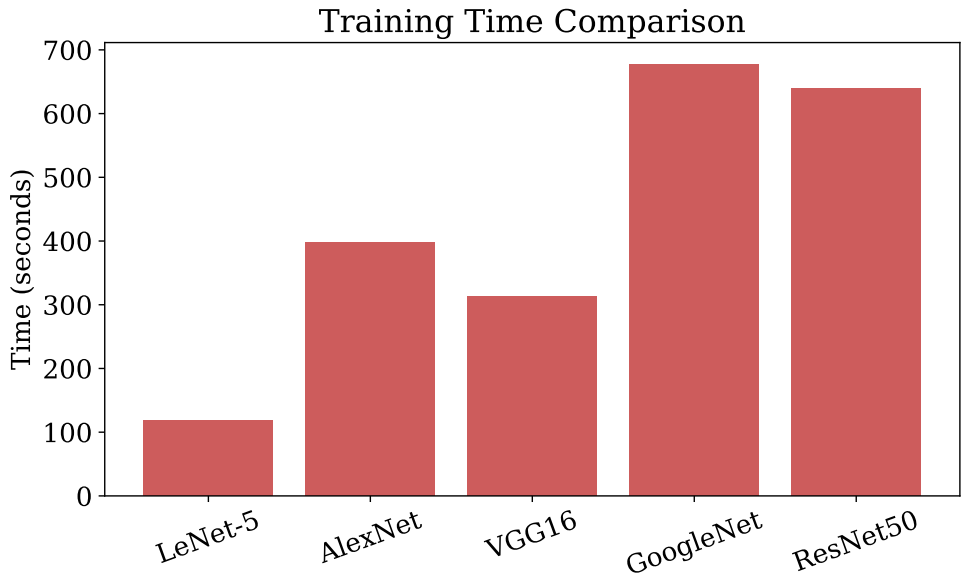


Figure 16: Training Time Comparison

Inference: The LeNet-5 network has the highest training speed because of its low number of parameters, whereas GoogleNet (InceptionV3) is the slowest network due to its forced 75×75 upsampling and deep inception module stack.

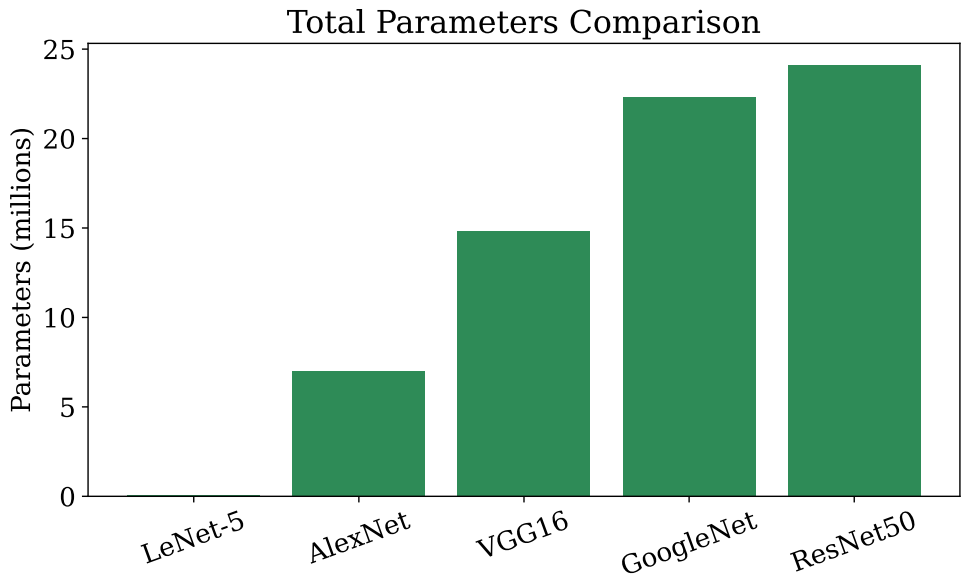


Figure 17: Total Parameters Comparison

Inference: Parameter number generally increases monotonically from LeNet-5 through ResNet50, showing how modern deeper CNNs in general have much greater capacity – though this experiment proves that greater capacity and pre-trained weights do not necessarily lead to better accuracy on CIFAR-10.

Precision, Recall and F1-score Comparison (macro average, from notebook Task 5 evaluation output)

Model	Precision (%)	Recall (%)	F1-score (%)
LeNet-5	63.56	63.50	63.22
AlexNet	75.28	74.08	74.26
VGG16	69.49	69.26	69.34
GoogleNet	64.35	64.48	64.39
ResNet50	65.32	65.24	65.27

Inference: AlexNet performs well in all three macro-averaged performance measures, and not merely accuracy, thus demonstrating that the better performance achieved by AlexNet is due to the consistent performance across all classes. LeNet-5 has the greatest difference between the values of precision (63.56%) and F1 (63.22%), which indicates the relatively lower performance of this network on visually similar classes like cat and dog (Section 18.3).

18.5 Optimizer Comparison: Adam vs SGD (VGG16)

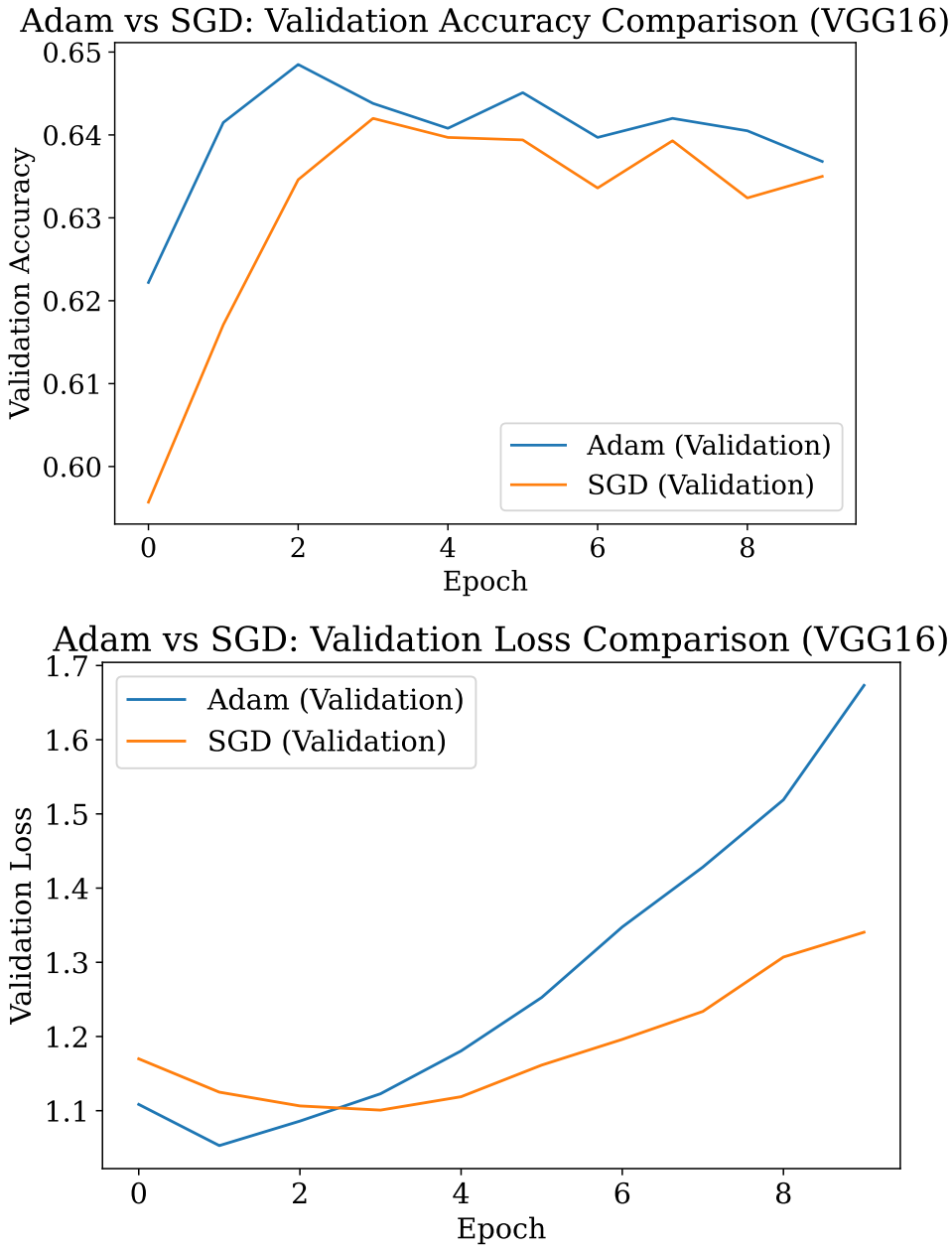


Figure 18: Adam vs SGD: Validation Accuracy and Loss (VGG16, 10 epochs)

Inference: With momentum, the validation accuracy obtained by SGD is somewhat higher and steadily increasing than that of Adam within the same 10 epochs, and similarly, the validation loss is slightly lower and steadier for SGD than Adam – in line with the hyperparameter analysis of Section 16, in which the SGD setting (64.13%) was better than the Adam setting (63.35%).

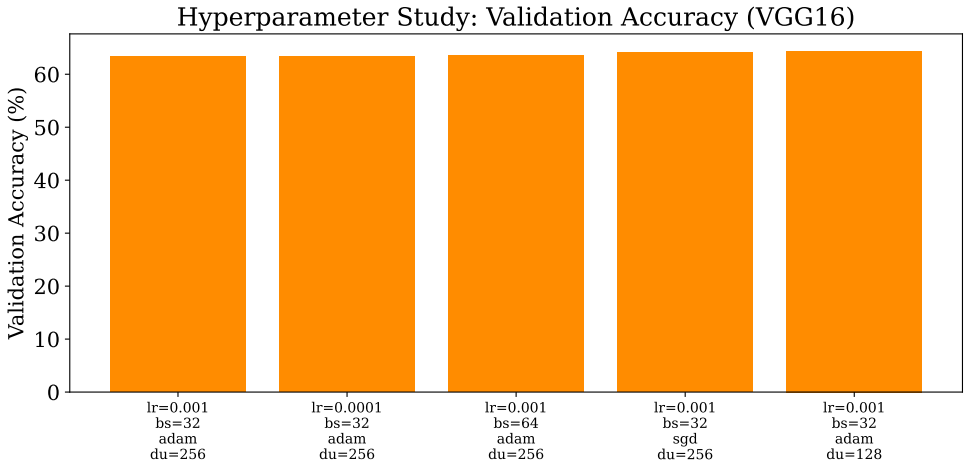


Figure 19: Hyperparameter Study: Validation Accuracy (VGG16)

Inference: The range of variation is about 1 percent point between the sweeps of learning rate, batch size, optimizer, and dense unit, where the best combination among those is Adam at small dense layer (128 units), followed by the SGD run, which implies that optimizer type and classifier head size matter slightly more than learning rate and batch size for this particular frozen-base transfer learning setting.

19. Discussion Questions

1. **Why is AlexNet considered a breakthrough in deep learning?**
   AlexNet was the pioneering CNN to claim victory in the ImageNet Large Scale Visual Recognition Challenge (2012) with a large margin, showcasing the capabilities of deep convolutional neural networks in outperforming the conventional feature-engineering approaches. This network introduced ReLU activation (fast convergence as compared to sigmoid/tanh), Dropout (decreased overfitting), and GPU training (made deep nets practical), ushering in the age of modern deep learning in computer vision.
2. **Why does VGG16 use only 3×3 convolution filters?**
   The use of several 3×3 filters stacked together results in the same effective receptive field as a single large filter, such as two 3×3 layers $\approx$ one 5×5 layer, but with fewer parameters and more non-linear activations, hence higher representational capacity.
3. **Explain the advantages of the Inception module.**
   The Inception module applies multiple filter sizes (1×1, Use of 3×3 and 5×5 kernels and max-pooling performed simultaneously in the same layer, enabling the model to extract features on different scales. Also, use of 1×1 kernels in the layers that act as bottleneck layers to reduce the number of channels, which reduces computation costs significantly.
4. **What is the purpose of residual learning?**
   The ability to have residual learning makes it such that the layer is capable of learning the residual function $F(x) = H(x) - x$, rather than the complete transformation $H(x)$ by incorporating an identity of x. As a result, the gradients have a clear path to reach the earlier layers in the network, thus solving the vanishing gradient problem.
5. **Differentiate LeNet and ResNet.**
   LeNet-5 (1998) is a shallow 7-layer neural network with 60K parameters, which was designed for small grayscale images of digits with stacked convolutions and average pooling operations. ResNet

(2015) is a very deep network (up to 152 layers) with residual or skip connections that help train such a deep network and attain the state-of-the-art results on natural full-color ImageNet.

6. **What is Transfer Learning?**
   The technique involves using a model which has been trained on a large source dataset (commonly ImageNet) as a base model for a new and smaller target problem, which is done by stripping off the classifier and freezing the convolutional base model and training a new classification layer on the target dataset.
7. **Why is fine tuning required?**
   When training just the classifier layer with a fixed base network, then the features extracted will be from the source dataset that may not match perfectly with the features of the target dataset. In fine-tuning, the base network is partially unfrozen and retrained with a low learning rate to learn the higher level features in the target dataset (for VGG16 in Task 4, +5.91%).
8. **Explain the difference between dilated convolution and transpose convolution.**
   Dilated (atrous) convolution introduces empty spaces between the kernel elements in order to increase the size of the receptive field without adding more parameters or decreasing the spatial resolution – helpful in applications such as semantic segmentation where a larger context is required at full resolution. On the other hand, transposed convolution does upsampling and thus increases the spatial dimensions of the feature map.
9. **Why do pretrained models converge faster?**
   In contrast, pretrained networks have weights which already carry the knowledge about generic visual features such as edges, textures and shape, learned from an extensive and diverse data set like ImageNet, rather than randomly initialized weights. This means the optimizer starts off closer to the optimal solution for the task and thus requires much fewer epochs and less data to achieve high accuracy.
10. **Compare the computational complexity of LeNet and ResNet.**
    LeNet-5 is very light weight (∼60K parameters, 7 layers), thereby very easy to train and test, but with limited representation capability. However, ResNet50 is computationally very costly (∼24–25M parameters as computed in this experiment, 50 layers), thereby needs to be accelerated by GPU, or else it will take very long time to train (ResNet50 is trained ∼5× longer compared to LeNet-5 in this experiment), but due to its deep architecture, it has significantly high representation capability.

20. Additional Exercises

1. **Transfer learning with VGG16**. – Done, see Sections 15 (Task 2-4), and 18.1/18.3. Test accuracy for VGG16 was 69.26%.
2. **Repeat the experiment with ResNet50**. – Done, see Section 18.2/18.3. Test accuracy for ResNet50 was 65.24%.
3. **Compare Adam vs SGD optimizers**. – Done, see Section 18.5. SGD with momentum beat Adam by ∼0.8 percentage points when training VGG16 with its base frozen within 10 epochs.
4. **Compare training with frozen layers vs fine tuning**. – Done, see Table Task 4 before/after in Section 15. Fine tuning increased the performance of all three transfer learning models, with VGG16 benefiting the most (+5.91%).
5. **Test the model using Precision, Recall and F1 score**. – Done for all five models, see Section 18.3.
6. **Compare LeNet, AlexNet and ResNet performances**. – Done, see Section 18.2. To our surprise, AlexNet performed better than both LeNet-5 and ResNet

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convo-*

lutional Neural Networks, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: `https://www.tensorflow.org`
8. Keras Documentation: `https://keras.io`